Asembler

Pętle, tablice

Pętla while

- Schemat:
  - Zapisanie w rejestrze wartości początkowych
  - **Stworzenie etykiety określającej początek pętli**
  - Sprawdzenie warunku wyjścia
  - Skok warunkowy do etykiety wyjścia
  - Instrukcje wewnątrz ciała pętli
  - **Skok bezwarunkowy do początku pętli**

```
__asm poczatek:
__asm cmp ... ...;
__asm je / jge / jl / jle / jne ... koniec;

__asm jmp poczatek;
__asm koniec:
```

```
short i, j, k;

i = 7; j = 9;
__asm   mov ax,0;
__asm   mov cx,j;
__asm   start:
__asm   cmp cx,0;
__asm   je koniec;
__asm   add ax,i;
__asm   sub cx,1;
__asm   jmp start;
__asm   koniec:
__asm   mov k, ax;
```

```
while (j <= 9) {
    i += i;
}
```

Pętla for

```
int i;
__asm mov eax, 0;
__asm mov ecx, 10;
__asm poczatek:
__asm add eax, ecx;
__asm loop poczatek;
__asm mov i, eax;


cout << i << endl;
```

```
int suma=0;
for(int i=10; i!=0; --i)
  suma += i;
```

Operatory i dyrektywy MASM

- OFFSET zwraca odległość zmiennej od początku otaczającego ją segmentu.
- PTR operator pozwala zastąpić domyślny rozmiar zmiennej..
- TYPE operator zwraca rozmiar (w bajtach) każdego elementu w tablicy.
- LENGTHOF operator zwraca liczbę elementów w tablicy.
- SIZEOF operator zwraca liczbę bajtów używanych przez inicjator tablicy.

Dyrektywa byte ptr[]

Pointer(ptr)

- Dyrektywa **byte ptr[]** pozwala odwołać się do bajta pod określonym adresem
  np. przez rejestr
- Wskaźnik na bajt

type PTR expression

QWORD PTR **[ecx]**	**64-bity**
DWORD PTR **dane**	**32-bity**
WORD PTR **num[0]**	**16-bitów**
BYTE PTR **dane**	**8-bitów**

```
.DATA
        num DWORD 0
.CODE
        mov ax, WORD PTR [num] ; Load a word-size value from a DWORD
```

Dyrektywa byte/word/dword ptr

Debug | x86 | Lokalny debuger Windows | Auto

ation1.cpp

plication1 | (Globalny zasięg)

```
#include <iostream>
char wek[5] = { 10,20,30,40,50 };
int wynik;

int main()
{
    __asm {
        xor eax, eax;
        mov ebx, offset wek ; adres początku tablicy
        inc ebx             ; ustawienie na drugi element
        mov al,byte ptr[ebx]; pobranie wartości tablicy
        mov wynik, eax      ; zapamiętanie wyniku
    }
    std::cout << wynik << "\n";
}
```

Konsola debugowania progra

```
20
```

Pamięć 1

Adres: 0x0068C034

```
0x0068C034  0a 14 1e 28 32 00 00 00
0x0068C052  00 00 00 00 00 00 00 00
```

array

Rejestry

```
EAX = 00000000 EBX = 00986000 ECX = 00000000 EDX = 00D6D590 ESI = 00681023 EDI = 00AFFB6C
  EIP = 00681E39 ESP = 00AFFAA0 EBP = 00AFFB6C EFL = 00000246
```

Rejestry

```
EAX = 00000000 EBX = 0068C034 ECX = 000
  EIP = 00681E3E ESP = 00AFFAA0 EBP = 0
```

Rejestry

```
EAX = 00000000 EBX = 0068C035 ECX = 0
  EIP = 00681E3F ESP = 00AFFAA0 EBP =
```

Rejestry

```
EAX = 00000014 EBX = 0068C035 ECX
  EIP = 00681E41 ESP = 00AFFAA0 EB
```

Dyrektywa dword ptr[]

```
#include <iostream>
int wek[5] = { 10,20,30,40,50 };
int wynik;

int main()
{
    __asm {
        xor eax, eax;
        mov ebx, offset wek   ; adres początku tablicy
        add ebx,8             ; ustawienie na trzeci element
        mov eax,dword ptr[ebx]; pobranie wartości tablicy
        mov wynik, eax        ; zapamiętanie wyniku
    }
    std::cout << wynik << "\n";
}
```

Pamięć 1

Adres: 0x00B6C034

```
0x00B6C034  0a 00 00 00 14 00 00 00 1e 00 00 00 28 00 00 00 32 00 00 00 00 00 00
```

dane 4 bajtowe

Konsola debugowania progra

```
30
```

Wskaźniki, adres zmiennej

```
#include <iostream>
unsigned int k;
int main()
{
    __asm {
        mov eax, offset k;
        mov ebx, 7;
        mov [eax], ebx;
    }
    std::cout << k;
}
```

```
Konsola debugowania progra
7
C:\vs2019_c\arch_komp\tabl
```

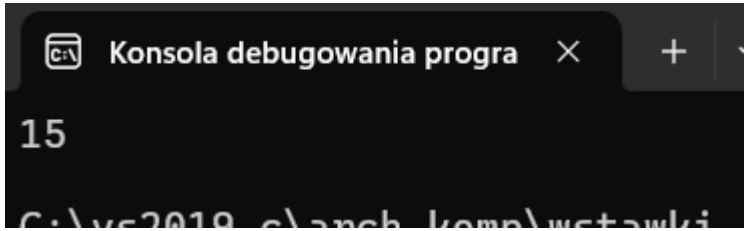
Tablica, wektor

```
#include <iostream>
#define N 5
const char k = N;
char tab[N] = { 1,2,3,4,5 };
unsigned short suma;
int main()
{

    __asm {
        xor eax, eax;
        mov cl, 0;
        mov dh, 0;
        mov ebx, offset tab;
    start:
        mov dl, byte ptr[ebx];
        add ax, dx;
        inc ebx;
        inc cl;
        cmp k, cl;
        jg start;
        mov suma, ax;
    }
    std::cout << suma;

}
```

```
Konsola debugowania progra

15
C:\vs2019_c\arch_komp\tab
9238392
```

Tablica, wektor – inne rozwiązanie

```
1   #include <iostream>
2
3   char tab[5] = {1,2,3,4,5};
4   unsigned short suma;
5
6   int main()
7   {
8       __asm
9       {
10          xor eax, eax;
11          xor ecx, ecx;
12          xor edx, edx;
13          mov ebx, offset tab;
14          mov ecx, 5;
15
16      et1:
17          mov dl, byte ptr[ebx];
18          add ax, dx;
19          inc ebx;
20          loop et1;
21          mov suma, ax;
22      }
23      std::cout << suma << " \n";
24  }
```



pętla

Zadania - pętle

1. Niech **i**, **k** będą zmiennymi typu **short int**. Napisz wstawkę asemblerową liczącą w pętli sumę liczb 1,2,3,...,i. Zapisz tą liczbę w zmiennej **k**.
2. Niech **i**, **j**, **k** będą zmiennymi typu **short int**. Napisz wstawkę asemblerową liczącą w pętli działanie potęgowania $\mathbf{i}^{\mathbf{j}}$. Zapisz tą liczbę w zmiennej **k**.
3. Niech **i**, **j** będą zmiennymi typu **unsigned int**. Napisz wstawkę asemblerową przepisującą do **j** wartość spod adresu zmiennej **i**.

Zadania - tablice

1. Napisz wstawkę asemblerową zapisującą do tablicy kolejne liczby naturalne od 0 do 10.
2. Napisz wstawkę asemblerową sumującą co drugą wartość z tablicy z poprzedniego zadania.
3. Napisz wstawkę asemblerową wyliczającą sumę wartość z elementów tablicy mniejszych od 5. Tablica składa się z liczb **char** / **int** /**short int**
4. Napisz wstawkę asemblerową wyliczającą ile razy w tablicy **TAB** występuje element **EL**. Wynik zapisz do zmiennej **k** typu **short int**
5. Dany jest **10** elementowy wektor danych typu **int** i wartość **k**. Napisz wstawkę asemblerową wyliczającą iloczyn wartości wektora mniejszych od wartości **k**.
6. Dany jest **10** elementowy wektor danych typu **int** i wartość **k**. Napisz wstawkę asemblerową wskazującą nr komórki wektora, kiedy pierwszy raz wystąpiła wartość **k**.